



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Патрик Барши

Статичка анализа Go кода у циљу читљивости и одржавања

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Патрик Барши
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Статичка анализа Go кода у циљу читљивости и одржавања
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	7/41/12/6/50/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	статичка анализа, линтер, Go, Rust, Tree-sitter, конкретно стабло синтаксе, дијагностике
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	У овом раду представљени су дизајн и имплементација алата за статичку анализу изворног кода написаног у програмском језику Go. Алат је имплементиран у програмском језику Rust и користи Tree-sitter као позадински систем за парсирање, чиме се добија конкретно стабло синтаксе анализираног кода. За разлику од анализатора интегрисаних у компајлер, овај приступ одржава алат независним од Go инфраструктуре. Имплементирано је 21 правило статичке анализе подељено у две категорије: општа правила која детектују честе програмерске грешке и правила изведена из књиге <i>100 Go Mistakes and How to Avoid Them</i> аутора Теиве Харсањија. Свако правило независно обрађује стабло синтаксе и производи дијагностике са прецизним информацијама о позицији у изворном коду. Алат је доступан као самостална апликација командне линије способна да анализира појединачне фајлове или целе директоријуме.
Датум прихватања теме, ДП:	01.01.2025
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
	Члан, ментор: Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Patrik Barši
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	Static Go code analysis for readability and maintainability
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/41/12/6/50/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	static analysis, linter, Go, Rust, Tree-sitter, concrete syntax tree, diagnostics
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents the design and implementation of a static analysis tool for source code written in the Go programming language. The tool is implemented in Rust and uses Tree-sitter as its parsing backend to produce a Concrete Syntax Tree of the analyzed code. Unlike compiler-integrated analyzers, this approach keeps the tool decoupled from the Go toolchain. The implementation covers 21 linting rules divided into two categories: general rules targeting common programming mistakes, and rules derived from the book <i>100 Go Mistakes and How to Avoid Them</i> by Teiva Harsanyi. Each rule operates independently on the syntax tree and emits diagnostics with precise source locations. The tool is provided as a standalone command-line application capable of analyzing individual Go source files or entire directory trees.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • **ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА**
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
2	Стање у области	3
2.1	go vet	3
2.2	staticcheck	3
2.3	revive	3
2.4	golangci-lint	3
2.5	Поређење постојећих алата	3
3	Технологије	5
3.1	Програмски језик Rust	5
3.2	Tree-sitter	5
3.3	Конкретно наспрам апстрактног стабла синтаксе	6
3.4	Библиотека clar	7
4	Архитектура	9
4.1	Компоненте система	9
4.2	Интерфејс правила	9
4.3	Обилазак конкретног стабла синтаксе	10
4.4	Правила анализе	10
4.5	Излазни формат	11
5	Имплементација правила	13
5.1	Општа правила добре праксе	13
5.2	Правила инспирисана књигом <i>100 Go Mistakes and How to Avoid Them</i>	17
6	Евалуација	25
6.1	Методологија	25
6.2	Резултати евалуације	25
6.3	Поређење са постојећим алатима	26
6.4	Преостало ограничење: правило ignored_error	28
6.5	Евалуација на стварном коду	28
7	Закључак	29
	Списак листинга	31
	Списак табела	33
	Списак коришћених скраћеница	35
	Списак коришћених појмова	37
	Биографија	39
	Литература	41

Go је програмски језик опште намене развијен у компанији Google, намењен за изградњу поузданих и ефикасних серверских система. Иако је компајлер строг у погледу основних семантичких правила, читав низ честих грешака и лоших пракси остаје изван опсега компајлера. Статичка анализа кода (енг. *static analysis*) је техника аутоматског испитивања изворног кода без његовог извршавања. Алати за статичку анализу, познати као линтери (енг. *linters*), откривају потенцијалне грешке, кршења конвенција и антио-брасце непосредно из изворног текста програма.

Постојећи линтери за Go (`go vet`, `staticcheck`, `revive`) у потпуности зависе од Go компајлерске инфраструктуре. Употребљавају апстрактно стабло синтаксе добијено директно из компајлера и захтевају да пројекат буде компајлабилан. Ова зависност онемогућава анализу некомпајлабилног кода и везује алате за Go алатни ланац.

У оквиру овог рада развијен је линтер за Go назван `golint`. Алат је имплементиран у програмском језику Rust и заснован је на библиотеци `Tree-sitter`, која производи конкретно стабло синтаксе директно из изворног текста без потребе за Go компајлером. Имплементирано је 21 правило подељено у два скупа: општа правила добре праксе и правила инспирисана књигом *100 Go Mistakes and How to Avoid Them* [1].

Рад је организован на следећи начин: у поглављу 2 дат је преглед постојећих алата за статичку анализу Go кода. У поглављу 3 описане су кључне технологије коришћене у имплементацији. У поглављу 4 приказана је архитектура алата, а у поглављу 5 детаљна имплементација свих правила. У поглављу 6 изведена је евалуација тачности алата. Поглавље 7 доноси закључак и правце даљег развоја.

Статичка анализа кода (енг. *static analysis*) представља технику испитивања изворног кода без његовог извршавања. За разлику од динамичке анализе, која прати понашање програма током извршавања, статичка анализа открива потенцијалне грешке, кршења стилских конвенција и лоше праксе непосредно из изворног кода програма. Алати који обављају ову врсту анализе називају се линтери (енг. *linters*).

У екосистему програмског језика Go постоји неколико устаљених алата за статичку анализу. У наставку су описани најзначајнији, уз фокус на начин на који приступају анализи кода.

2.1 go vet

`go vet` је алат за статичку анализу уграђен директно у стандардну инсталацију Go језика. Анализира изворни код употребом апстрактног стабла синтаксе (АСС, енг. *Abstract Syntax Tree*) и информација о типовима добијених од Go компајлера. Фокусиран је на детектовање конструкција које су синтаксно исправне, али врло вероватно погрешне (на пример, погрешан формат аргумената у позивима функција попут `fmt.Printf`, или поређење структура које не смеју бити поређене).

Будући да је `go vet` везан за Go компајлер, не може се користити независно од Go инфраструктуре [2].

2.2 staticcheck

`staticcheck` је тренутно најпопуларнији самостални линтер за Go. Покрива широк спектар провера: од откривања грешака и неискоришћеног кода до дијагностика специфичних за поједине пакете из стандардне библиотеке. Такође користи АСС и информације о типовима из Go компајлера, чиме постиже висок степен прецизности [3].

2.3 revive

`revive` је конфигурабилан алат за статичку анализу Go кода са нагласком на проверу стила и добрих пракси. Нуди могућност укључивања и искључивања појединачних правила, као и подршку за прилагођена правила путем додатака (енг. *plugins*). Такође зависи од Go компајлерске инфраструктуре [4].

2.4 golangci-lint

`golangci-lint` није самосталан линтер, већ збирка (енг. *aggregator*) која обједињује велики број постојећих Go линтера, укључујући `staticcheck` и `revive`, и омогућава њихово паралелно извршавање уз централизовану конфигурацију. Широко је коришћен у CI/CD окружењима [5].

2.5 Поређење постојећих алата

Сви наведени алати ослањају се на АСС и информације о типовима добијене из Go компајлера. Ово им омогућава висок степен прецизности, али их чини нераздвојиво везаним за Go инфраструктуру. Нису употребљиви изван Go екосистема и не могу се лако прилагодити другим језицима.

Табела 1: Поређење алата за статичку анализу Go кода

Алат	Тип стабла	Независан од Go	Проширив
go vet	ACC	Не	Ограничено
staticcheck	ACC	Не	Не
revive	ACC	Не	Да
golangci-lint	Н/А	Не	Да
golint (овај рад)	KCC	Да	Да

Из табеле [1](#) може се уочити да се алат развијен у оквиру овог рада разликује по употреби конкретног стабла синтаксе (KCC) и независности од Go компајлерске инфраструктуре, о чему је детаљније реч у поглављу [4](#).

У овом поглављу описане су кључне технологије коришћене при изради алата: програмски језик Rust, библиотека за парсирање Tree-sitter, концепти конкретног и апстрактног стабла синтаксе, и библиотека `clap` за обраду аргумената командне линије.

3.1 Програмски језик Rust

Rust је системски програмски језик опште намене [6], [7]. Главна особина по којој се Rust издваја међу системским језицима јесте гаранција безбедности меморије без употребе сакупљача смећа (енг. *garbage collector*).

Безбедност меморије постиже се кроз систем власништва (енг. *ownership*) и позајмљивања (енг. *borrowing*). Компајлер прати власника сваке вредности у меморији и аутоматски ослобађа меморију када власник изађе из опсега. Систем позајмљивања прецизно одређује колико референци може постојати у исто време. Дозвољен је произвољан број непроменљивих (енг. *immutable*) референци или тачно једна променљива (енг. *mutable*) референца, никада обоје истовремено. Ова правила компајлер проверава у потпуности при превођењу, а не у току извршавања [8]. Резултат је елиминација читаве класе грешака (употреба ослобођене меморије, енгл. *use-after-free*; трка података, енгл. *data race*; нулти показивачи) без додатних трошкова у току извршавања.

Rust је изабран за имплементацију алата из неколико разлога. Прво, алат потенцијално обрађује велики број фајлова, па је перформанса важна. Будући да се Rust компајлира директно у машински код, не постоји трошак покретања виртуелне машине нити паузе сакупљача смећа. Друго, систем власништва онемогућава читаву класу имплементационих грешака унутар самог алата, чиме је обезбеђена поузданост. Треће, Rust нуди богат екосистем библиотека преко регистра `crates.io`, међу којима су Tree-sitter и `clap` библиотеке употребљене у овом пројекту.

3.2 Tree-sitter

Tree-sitter је библиотека за инкрементално парсирање изворног кода [9]. Данас се употребљава у широком спектру програмских едитора и алата, укључујући Neovim, Emacs и GitHub интерфејс, за истицање синтаксе, навигацију кодом и структурно претраживање.

Систем се састоји из два дела: генератора парсера и извршне библиотеке (енг. *runtime*). Генератор прима формалну граматiku одређеног програмског језика и из ње производи парсер у програмском језику C. Извршна библиотека читава тај парсер у току рада програма и употребљава га за парсирање изворног текста. Захваљујући овој архитектури, Tree-sitter је у потпуности независан од конкретног програмског језика; за сваки подржани језик постоји засебан пакет са граматиком.

У пројекту су коришћена два Rust пакета: `tree-sitter`, који садржи извршну библиотеку са API-јем за Rust, и `tree-sitter-go`, који садржи граматiku за програмски језик Go.

Важна особина Tree-sitter-а је подршка за инкрементално парсирање: при малој промени изворног кода библиотека репарсира само измењене делове стабла уместо целог документа. Иако ова особина није искоришћена у алату за статичку анализу, пошто се фајлови читају само по један пут, она је главни разлог широке употребе Tree-sitter-а у едиторима кода где је брзина одговора кључна.

3.3 Конкретно наспрам апстрактног стабла синтаксе

Оба облика стабла синтаксе представљају синтаксну структуру програма у виду стабла, али садрже различит ниво детаља.

Апстрактно стабло синтаксе (АСС, енг. *Abstract Syntax Tree*) уклања из стабла синтаксну „буку“, тј. кључне речи, интерпункцију и размаке, и чува само семантички значајне елементе. На пример, у изразу `a + b` АСС садржи чвор за бинарну операцију са два потомка (`a` и `b`), а оператор `+` је имплицитан у типу чвора, а не засебан чвор. Алати попут `go vet` и `staticcheck` раде над АСС-ом добијеним директно од Go компајлера, уз информације о типовима, чиме добијају пуну семантичку слику програма. Последица је, чврста зависност од Go инфраструктуре: `go vet` је уграђен у Go ланац алата, а `staticcheck` захтева да пројекат буде компајлабилан.

Конкретно стабло синтаксе (КСС, енг. *Concrete Syntax Tree*) производи Tree-sitter, чува сваки токен изворног кода: заграде, зарезе, кључне речи и тачке-зарезе. Ово стабло представља целу репрезентацију изворног кода. Из КСС-а је могуће реконструисати оригинални текст у потпуности. Сваки чвор у стаблу носи прецизну позицију у изворном коду (ред и колону), па је могуће детектовати проблем и пријавити га кориснику са тачном локацијом.

Листинг 1 илуструје КСС структуру коју Tree-sitter производи за функцију `func add(a, b int) int { return a + b }`.

```
(source_file
  (function_declaration
    name: (identifier)          // "add"
    parameters: (parameter_list
      "("
      (parameter_declaration
        name: (identifier)      // "a"
        name: (identifier)      // "b"
        type: (type_identifier)) // "int"
      ")")
    result: (type_identifier)    // "int"
    body: (block
      "{"
      (return_statement
        "return"
        (binary_expression
          left: (identifier)     // "a"
          right: (identifier)))  // "b"
      "}")
    )))
```

Листинг 1: КСС структура Tree-sitter парсера за кратку Go функцију

Tree-sitter разликује именоване (енг. *named*) и анонимне (енг. *anonymous*) чворове. Именовани чворови, попут `function_declaration`, `parameter_list` или `binary_expression`, одговарају граматичким нетерминалима и носе семантичку тежину. Анонимни чворови представљају литералне токене из граматике, попут `"("`, `"{"` или `"return"`, и приказани су под наводницима у листингу 1. Употребом методе `named_children` API-ја могу се добити само именовани потомци чвора, чиме се поједностављује обилазак стабла у правилима где анонимни чворови нису потребни.

Употреба КСС-а носи са собом значајну предност: алат не захтева ниједну компоненту Go инфраструктуре нити информације о типовима. Анализа се обавља искључиво над синтаксном структуром фајла, без претходног компајлирања пројекта.

3.4 Библиотека clap

clap (енг. *Command Line Argument Parser*) је Rust библиотека за декларативну дефиницију аргумената командне линије [10]. Уместо ручног парсирања низа аргумената, програмер дефинише структуру са атрибутима, а clap аутоматски генерише парсер, поруке о грешкама и помоћни текст при позиву са заставицом `--help`.

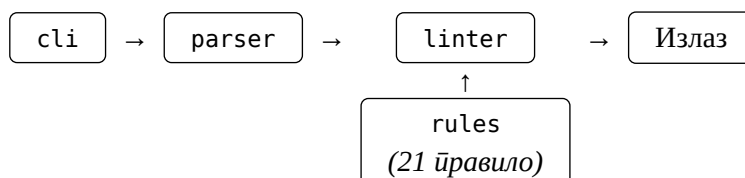
```
#[derive(Parser, Debug)]
#[command(name = "golint")]
#[command(about = "Lints Go source files")]
pub struct Args {
    #[arg(value_name = "PATH")]
    pub path: PathBuf,

    #[arg(short, long, value_name = "FILE")]
    pub output: Option<PathBuf>,
}
```

Листинг 2: Дефиниција аргумената употребом clap библиотеке

Атрибут `#[derive(Parser)]` инструктира clap да аутоматски генерише имплементацију парсера за дату структуру. Поље `path` је обавезан позициони аргумент који прима путању до Go фајла или директоријума. Атрибут `#[arg(short, long)]` на пољу `output` означава да је реч о опционом именованом аргументу доступном у облику `-o FILE` или `--output FILE`. Ако аргумент није наведен, вредност је `None` и дијагностике се исписују на стандардни излаз.

Алат је организован као вишефазни пајплајн за статичку анализу. Свака фаза обавља јасно дефинисан задатак и предаје свој излаз следећој фази, чиме је постигнута модуларност и могућност независног тестирања сваке компоненте, као што је илустровано на слици 3.



Листинг 3: Архитектура алата за статичку анализу

4.1 Компоненте система

Систем се састоји од пет основних компоненти, организованих у одвојене Rust модуле.

cli: обрађује аргументе командне линије употребом библиотеке `clap` [10]. Прима два параметра: обавезну путању до Go фајла или директоријума, и опциони параметар `--output` за писање дијагностика у фајл уместо на стандардни излаз.

parser: обавља парсирање Go изворног кода употребом библиотеке `Tree-sitter`. Модул излаже јединствену функцију `parse`, која прима путању до фајла, учитава садржај са диска, иницијализује `Tree-sitter` парсер са Go граматиком и враћа пар `(Tree, String)` чији су чланови конкретно стабло синтаксе и оригинални текст кода. Текст кода је неопходан јер КСС чува само позиције токена, а не и текстуалне вредности.

linter: агрегира сва правила и управља процесом анализе. Структура `Linter` садржи вектор показивача на правила `(Vec<Box<dyn Rule>>)`. Метода `run` прима стабло, изворни текст и путању фајла, покреће свако правило над тим стаблом и акумулира све дијагностике у заједнички вектор.

rules: скуп од 21 независних модула, сваки имплементира трејт `Rule`. Правила су подељена у два скупа: општа правила добре праксе и правила инспирисана књигом *100 Go Mistakes and How to Avoid Them* [1]. Сваки модул је независан, тако да додавање или уклањање правила не захтева измене у осталим деловима система.

diagnostics: дефинише структуру `Diagnostic` која чува информације о откривеном проблему: путању до фајла, редни број реда и колоне (нумерисани од 1), идентификатор правила и описну поруку. Конструктор аутоматски конвертује `Tree-sitter`-ову нулу-индексирану позицију `(Point)` у уобичајену нумерацију.

4.2 Интерфејс правила

Свако правило имплементира Rust трејт (енг. *trait*) приказан на листингу 4:

```
pub trait Rule {  
    fn name(&self) -> &'static str;  
    fn check(&self, tree: &Tree, source: &str, file: &str) -> Vec<Diagnostic>;  
}
```

Листинг 4: Дефиниција `Rule` трејта

Метода `name` враћа статички стринг са идентификатором правила, употребљеним у излазу дијагностике. Метода `check` прима три параметра: КСС стабло, изворни текст и путању до фајла. Враћа вектор детектованих проблема.

Захваљујући овом дизајну, додавање новог правила своди се на креирање новог модула и регистравање у листи унутар структуре Linter. Постојећа правила остају потпуно непромењена.

4.3 Обилазак конкретног стабла синтаксе

Свако правило добија приступ целом КСС стаблу и оригиналном изворном тексту. Задатак правила је да пронађе синтаксне конструкције које одговарају одређеном обрасцу грешке и за сваку такву конструкцију произведе дијагностику са тачном позицијом у коду.

Правила бирају одговарајући приступ обиласку у зависности од сложености обрасца који траже. Неки обрасци захтевају само линеарни преглед непосредне деце одређеног чвора, сложенији захтевају рекурзивно претраживање у дубину, а поједина правила примењују вишефазни поступак у коме се прво прикупљају декларације па затим проверава употреба. Детаљи конкретних приступа описани су у поглављу 5.

4.4 Правила анализе

Имплементирана су укупно 21 правила, подељена у две категорије. Табела 2 садржи општа правила добре праксе, а табела 3 правила инспирисана књигом *100 Go Mistakes and How to Avoid Them*.

Табела 2: Општа правила добре праксе

Идентификатор	Опис
empty_block	Детекција празних блокова кода
unreachable_code	Код иза return, break, continue, fallthrough или panic никад се не извршава
ignored_error	Грешка додељена бланк идентификатору _ тихо се занемарује
unused_variable	Променљива је декларисана, али се нигде не употребљава
import_rules	Детекција дубликата увоза и некоришћених увоза
variable_shadowing	Локална декларација прекрива променљиву из спољашњег опсега
defer_in_loop	defer унутар петље одлаже ослобађање ресурса до повратка из функције

Табела 3: Правила инспирисана књигом *100 Go Mistakes and How to Avoid Them*

Идентификатор	Опис
utility_packages	Пакет носи превише генеричко име (нпр. <code>utils</code> , <code>helpers</code> , <code>common</code>)
no_any	Употреба типа <code>any</code> или празног интерфејса <code>interface{}</code>
generics_usage	Позив метода на генеричком параметру типа уместо на конкретном типу
mutex_embedding	Уграђени <code>sync.Mutex</code> у структури открива методе закључавања јавном API-ју
package_collision	Локална променљива прекрива име увезеног пакета у том опсегу
integer_overflow	Целобројни литерал прелази опсег декларисаног типа
slice_initialization	<code>make([]T, 0)</code> без капацитета изазива поновљена реалоцирања при расту исечка
nil_empty_slice	Иницијализација исечка са <code>[]T{}</code> ; нил исечак је пожељнији ако елементи нису познати унапред
slice_empty_check	Поређење исечка са <code>nil</code> не покрива случај непразног исечка нулте дужине
slice_copy	Одредишни исечак у позиву <code>copy</code> има нулту дужину, па се ниједан елемент не копира
slice_append_side_effect	Додавање у подисечак без ограничења капацитета може изменити оригинални исечак
range_copy_modification	Измена поља <code>range</code> елемента мења копију, а не оригинал у исечку
range_pointer_capture	Узимање адресе <code>range</code> променљиве у свакој итерацији даје исти показивач
string_rune	<code>len</code> или индексирање над бајт-конвертованим стрингом броји бајтове, не Unicode знакове

4.5 Излазни формат

Свака детектована дијагностика исписује се у следећем формату:

```
<фајл>:<ред>:<колона> [<правило>] <порука>
```

Листинг 5: Формат излазне дијагностике

Пример стварног излаза приказан је на листингу 6.

```
main.go:14:3 [defer_in_loop] defer statement inside for loop may delay resource release
main.go:27:9 [unused_variable] Unused variable `err`
main.go:41:5 [ignored_error] Error value assigned to blank identifier
```

Листинг 6: Пример излаза алата

Ако је наведен параметар `--output`, дијагностике се уписују у задати фајл уместо на стандардни излаз, чиме је омогућена интеграција у CI/CD окружења и друге аутоматизоване токове рада.

Глава 5

Имплементација правила

У овом поглављу описана је имплементација свих 21 правила алата. Свако правило имплементира трејт Rule и добија приступ КСС стаблу и оригиналном изворном тексту. Правила су подељена у два скупа: општа правила добре праксе, описана у овом поглављу, и правила инспирисана књигом *100 Go Mistakes and How to Avoid Them* [1], описана у следећем поглављу.

5.1 Општа правила добре праксе

5.1.1 Празан блок кода (empty_block)

Празан блок кода је блок са телом без наредби. Иако је синтаксно исправан, углавном представља грешку или заборављен код.

```
if err != nil {  
    // заборављена обрада грешке  
}
```

Листинг 7: Пример празног блока кода

Правило обилази стабло рекурзивним обиласком у дубину и за сваки чвор типа block броји именоване потомке употребом методе named_child_count. Ако је тај број нула, блок је празан и генерише се дијагностика.

```
if node.kind() == "block" {  
    let named = node.named_child_count();  
    if named == 0 {  
        diagnostics.push(Diagnostic::new(  
            file, "Empty code block detected",  
            EmptyBlock.name(), node.start_position(),  
        ));  
    }  
}
```

Листинг 8: Провера броја именованих потомака чвора block

5.1.2 Недостижни код (unreachable_code)

Наредбе које следе после безусловне терминирајуће наредбе (return, break, continue, fallthrough или позив panic) никада неће бити извршене.

```
func f() {  
    return  
    fmt.Println("никада се не извршава")  
}
```

Листинг 9: Пример недостижног кода

Правило прегледа сваки чвор statement_list и линеарно пролази кроз именоване потомке одржавајући заставицу found_terminator. Чим се наиђе на терминирајућу наредбу, заставица се поставља и свака наредна наредба у истом листу добија дијагностику. Позив panic се препознаје тако што се провери да ли је чвор expression_statement чији именовани потомак је call_expression са именом функције panic.

```

let mut found_terminator = false;
for i in 0..node.named_child_count() {
    if let Some(stmt) = node.named_child(i as u32) {
        if found_terminator {
            diagnostics.push(/* дијагностика */);
            continue;
        }
        if Self::is_terminating(stmt, source) {
            found_terminator = true;
        }
    }
}

```

Листинг 10: Линеарни пролаз кроз листу наредби са заставицом

5.1.3 Игнорисана грешка (ignored_error)

У Go-у, функције које могу да генеришу грешку по конвенцији враћају вредност типа `error` као последњи повратни аргумент. Додељивање те вредности бланк идентификатору `_` тихо је занемарује.

```
val, _ := strconv.Atoi("abc") // грешка конверзије се игнорише
```

Листинг 11: Тихо игнорисање грешке бланк идентификатором

Правило обилази чворове `short_var_declaration` и `assignment_statement` и прегледа све идентификаторе на левој страни доделе (поље `left`). Ако неки идентификатор има текстуалну вредност `_`, генерише се дијагностика. Правило пријављује сваки бланк идентификатор, без обзира на тип вредности коју прима, будући да информације о типовима нису доступне у приступу заснованом на КСС стаблу.

```

"short_var_declaration" | "assignment_statement" => {
    if let Some(lhs) = node.child_by_field_name("left") {
        for i in 0..lhs.named_child_count() {
            if let Some(id) = lhs.named_child(i as u32) {
                if id.utf8_text(source.as_bytes())
                    .unwrap_or("") == "_" {
                    diagnostics.push(/* дијагностика */);
                }
            }
        }
    }
}

```

Листинг 12: Провера левих операнда доделе на бланк идентификатор

5.1.4 Некоришћена променљива (unused_variable)

Декларисана, а никада коришћена променљива указује на заборављен код или логичку грешку. Иако Go компајлер већ одбија непакетске некоришћене променљиве, правило покрива и случајеве у оквиру истог `statement_list` чвора где компајлер не пријављује грешку.

```

func f() {
    x := 42 // x никада се не употребљава
    y := 10
    fmt.Println(y)
}

```

Листинг 13: Пример некоришћене променљиве

Имплементација је двофазна. У првој фази, пролази се кроз именоване потомке чвора `statement_list` и у хеш-мапу `declared` прикупљају се сви идентификатори декларисани путем `short_var_declaration` и `var_declaration`, уз искључивање бланк идентификатора `_`. У другој фази, рекурзивно се прикупљају сви идентификатори унутар блока у листу и бројеви јављања складиштени су у хеш-мапи `used`. Ако се

идентификатор јавља само један пут, то значи да се јавља искључиво на месту декларације и никада се не употребљава.

```
// Фаза 2: бројање јављања
let mut identifiers = Vec::new();
Self::collect_identifiers(node, source, &mut identifiers);
for name in identifiers {
    *used.entry(name).or_insert(0) += 1;
}
// Детекција: јавља се само у декларацији
for (name, decl_node) in declared {
    if used.get(&name).cloned().unwrap_or(0) <= 1 {
        diagnostics.push(/* дијагностика */);
    }
}
```

Листинг 14: Двофазна детекција некоришћених променљивих

5.1.5 Правила увоза (import_rules)

Правило детектује два проблема: дубликату увоза (исти пакет увезен двапут) и некоришћене увозе (пакет увезен, али нигде не употребљен).

```
import (
    "fmt"
    "fmt" // дубликат
    "os"   // некоришћен
)
```

Листинг 15: Дубликат и некоришћен увоз

У првом пролазу, рекурзивно се прикупљају сви чворови `import_spec`. Из сваког се извлачи путања пакета (чвор `interpreted_string_literal`) и локално име (чвор `identifier` ако постоји псеудоним, иначе последњи сегмент путање). У другом пролазу, прикупљају се сви чворови `selector_expression` и из поља `operand` читају имена употребљених пакета. На крају се проверава да ли је путања увоза виђена раније (дубликат) и да ли је локално име пакета присутно у скупу употребљених пакета (некоришћен увоз).

```
for (path, local_name, position) in imports {
    if seen.contains_key(&path) {
        diagnostics.push(/* дубликат */);
    } else {
        seen.insert(path.clone(), true);
    }
    if local_name != "_" && !used_packages.contains(&local_name) {
        diagnostics.push(/* некоришћен */);
    }
}
```

Листинг 16: Провера дубликата и некоришћених увоза

5.1.6 Засенчење променљиве (variable_shadowing)

Засенчење (енг. *variable shadowing*) настаје када унутрашњи опсег декларише нову променљиву истог имена као постојећа из спољашњег опсега. Оригинална вредност постаје недоступна унутар унутрашњег блока, што може проузроковати суптилне логичке грешке.

```

val := 10
if val > 5 {
    val := 20      // засенчује спољашњи val
    fmt.Println(val) // исписује 20
}
fmt.Println(val)   // исписује 10, не 20

```

Листинг 17: Пример засенчења променљиве

Правило одржава стек опсега: листу хеш-мапа, где свака хеш-мапа представља скуп декларисаних променљивих у одређеном опсегу. При уласку у нови `block` или `statement_list` чвор, прикупљају се све `short_var_declaration` и `var_declaration` декларације. За сваку нову декларацију, претражују се сви родитељски опсези; ако нека родитељска хеш-мапа садржи исто име, пријављује се засенчење. По завршетку обраде блока, тренутни опсег се скида са стека пре него што се настави обилазак суседних чворова.

```

for parent_scope in parent_scopes.iter().rev() {
    if parent_scope.contains_key(name) {
        diagnostics.push(Diagnostic::new(
            file,
            format!("Variable `{}` shadows variable \
                    from outer scope", name),
            VariableShadowing.name(),
            id.start_position(),
        ));
        break;
    }
}
current_scope.insert(name.to_string(), id);

```

Листинг 18: Провера засенчења претраживањем стека опсега

5.1.7 Одлагање ресурса унутар петље (`defer_in_loop`)

Наредба `defer` у Go-у одлаже извршавање функције до тренутка повратка из текуће функције, а не до краја тренутне итерације петље. Употреба `defer` унутар петље стога акумулира одложена позивања кроз све итерације и ослобађа ресурсе тек по изласку из функције, уместо одмах по завршетку сваке итерације.

```

for _, f := range files {
    defer f.Close() // f.Close се позива тек кад функција врати вредност
}

```

Листинг 19: Пример погрешне употребе `defer` унутар петље

Правило претражује стабло рекурзивним обилазком и за сваки чвор `for_statement` покреће засебан унутрашњи обилазак. Унутрашњи обилазак рекурзивно пролази кроз децу `for_statement` чвора и пријављује сваки `defer_statement` на који наиђе. Кад год је `defer_statement` пронађен директно у телу петље или у угнеженом блоку унутар те исте петље, генерише се дијагностика. Наиђе ли се на угнежден `func` литерал, обилазак у том подстаблу се не зауставља јер `defer` унутар угнеженог функцијског литерала припада опсегу те функције, а не спољашње петље.

```

fn check_for_loop(node: Node, file: &str,
                  diagnostics: &mut Vec<Diagnostic>) {
    for i in 0..node.child_count() {
        if let Some(child) = node.child(i as u32) {
            if child.kind() == "defer_statement" {
                diagnostics.push(/* дијагностика */);
            }
            Self::check_for_loop(child, file, diagnostics);
        }
    }
}

```

Листинг 20: Рекурзивна провера defer унутар for_statement чвора

5.2 Правила инспирисана књигом *100 Go Mistakes and How to Avoid Them*

5.2.1 Генеричко име пакета (utility_packages)

Пакети са именима попут `utils`, `common` или `helpers` окупљају несродне функционалности без јасне сврхе, чиме отежавају одржавање и разумевање кода [1].

```
package utils // превише генеричко име пакета
```

Листинг 21: Пример генеричког имена пакета

Правило проналази чвор `package_clause` у корену стабла, из њега чита `package_identifier` и проверава га у унапред одређеном скупу забрањених имена.

```

fn is_utility_name(name: &str) -> bool {
    let generic = ["utils", "util", "common",
                  "shared", "base", "helpers", "helper"];
    generic.contains(&name.to_lowercase().as_str())
}

```

Листинг 22: Листа забрањених имена пакета

5.2.2 Употреба типа `any` (no_any)

Тип `any` (псеудоним за `interface{}`) онемогућава статичку проверу типова и уноси у код несигурност карактеристичну за динамички типизоване језике [1].

```

func process(v any) { ... } // any у параметру
var cache map[string]interface{} ... // празан интерфејс

```

Листинг 23: Употреба `any` и `interface{}`

Правило обилази стабло и пријављује два случаја. Прво, сваки чвор `type_identifier` чија је текстуална вредност `any`. Друго, сваки чвор `interface_type` код кога ниједан именовани потомак није `method_spec` или `type_identifier`, тј. интерфејс са празним телом.

```

"type_identifier" => {
    if text == "any" { diagnostics.push(/* ... */); }
}
"interface_type" => {
    let empty = node.named_children(&mut cursor)
        .all(|c| c.kind() != "method_spec"
                && c.kind() != "type_identifier");
    if empty { diagnostics.push(/* ... */); }
}

```

Листинг 24: Детекција `any` и `interface{}`

5.2.3 Позив методе на генеричком параметру (generics_usage)

Када генеричка функција позива методу на параметру типа, то је знак да би конкретан тип директно испунио сврху без генерика [1].

```
func Print[T Stringer](v T) {
    fmt.Println(v.String()) // позив методе на генеричком T
}
```

Листинг 25: Позив методе на генеричком параметру типа

Имплементација је вишефазна. У првој фази из `type_parameter_list` чвора прикупљају се имена параметара типа. Ако се неки параметар јавља у повратном типу функције, искључује се из листе (та употреба генерика је оправдана). У другој фази, имена параметара из листе функцијских параметара повезују се са именима параметара типа. У трећој фази, тело функције се претражује за `selector_expression` чворове чији је `operand` идентификатор из те листе.

```
if node.kind() == "selector_expression" {
    if let Some(operand) = node.child_by_field_name("operand") {
        if let Ok(name) = operand.utf8_text(source.as_bytes()) {
            if params_to_type_param.contains_key(name) {
                diagnostics.push(/* ... */);
            }
        }
    }
}
```

Листинг 26: Детекција позива методе на генеричком аргументу

5.2.4 Уграђени мутекс у структури (mutex_embedding)

У Go-у, структура (енг. *struct*) може да угради (енг. *embed*) други тип тако што га наведе без имена поља. Уграђивање промовише сва јавна поља и методе угнежђеног типа на ниво спољашње структуре, тј. постају доступне директно преко вредности.

Мутекс (`sync.Mutex`) је примитив за синхронизацију у вишенитном програмирању: позив `Lock()` блокира нит до момента кад се ресурс ослободи позивом `Unlock()`. Ако се мутекс угради у структуру уместо да се декларише као именовано поље, методе `Lock` и `Unlock` постају видљиви јавни API те структуре. Свако ко поседује инстанцу може директно да је закључа или откључа, заобилазећи намеравани интерфејс, и тако наруши синхронизацију [1].

```
// Проблематично: Lock и Unlock су јавне методе Cache-a
type Cache struct {
    sync.Mutex
    data map[string]int
}
cache.Lock() // спољни позивалац може директно да закључа

// Исправно: мутекс је приватан, не излаже своје методе
type Cache struct {
    mu sync.Mutex
    data map[string]int
}
```

Листинг 27: Уграђени наспрам именованог `sync.Mutex`

Правило проналази чворове `struct_type` и унутар `field_declaration_list` тражи поља декларације која немају `field_identifier` потомка. Таква поља су уграђена (без имена). Ако је тип уграђеног поља `sync.Mutex` или `sync.RWMutex`, генерише се дијагностика.

```
// Уграђено поље нема field_identifier потомка
if !has_name {
    if let Some(t) = type_node {
        let text = t.utf8_text(source.as_bytes());
        if text == "sync.Mutex" || text == "sync.RWMutex" {
            diagnostics.push(/* ... */);
        }
    }
}
```

Листинг 28: Детекција уграђеног мутекса по одсуству имена поља

5.2.5 Сукоб имена са увезеним пакетом (package_collision)

Ако локална променљива носи исто име као увезени пакет, тај пакет постаје недоступан у том опсегу [1].

```
import "fmt"

func f() {
    fmt := "текст" // fmt пакет је сад недоступан
    fmt.Println() // грешка компајлирања
}
```

Листинг 29: Локална променљива прекрива увезени пакет

У првој фази прикупљају се локална имена свих увоза из `import_spec` чворова. У другој фази, рекурзивно се обилази стабло и проверавају `short_var_declaration`, `var_spec`, `const_spec` и `parameter_declaration` чворови. Ако је новодекларисано име присутно у скупу увезених пакета, пријављује се колизија.

```
if kind == "short_var_declaration" {
    // за сваки идентификатор на левој страни
    if imports.contains_key(name) {
        diagnostics.push(/* ... */);
    }
}
```

Листинг 30: Провера нових декларација наспрам скупа увезених имена

5.2.6 Целобројно прекорачење (integer_overflow)

Додељивање литералне вредности изван опсега декларисаног целобројног типа резултује прекорачењем [1].

```
var x int8 = 200 // int8 опсег је [-128, 127]
```

Листинг 31: Целобројни литерал изван опсега типа

Правило обилази чворове `var_spec` и `const_spec` и чита тип из поља `type`. Ако је тип познати целобројни тип, позива се рекурзивни евалуатор константних израза. Евалуатор подржава целобројне литерале, унарне операторе и бинарне аритметичке операторе употребом аритметике са провером прекорачења (`checked_add`, `checked_mul`...). Добијена вредност се упоређује са границама типа.

```
fn get_type_limits(t: &str) -> Option<(i128, i128)> {
    match t {
        "int8" => Some((-128, 127)),
        "int16" => Some((-32768, 32767)),
        "uint8" => Some((0, 255)),
        // ...
        _ => None,
    }
}
```

Листинг 32: Табела граница целобројних типова

5.2.7 Неефикасна иницијализација исечка (slice_initialization)

Позив `make([]T, 0)` без аргумента капацитета ствара исечак нулте дужине и нулте резервисане меморије. Свако накнадно додавање елемената изазива реалокацију, при чему Go удвостручује позадинске низ [1].

```
s := make([]int, 0)      // без капацитета
s = append(s, 1, 2, 3)  // вишеструке реалокације
```

Листинг 33: Иницијализација исечка без капацитета

Правило проналази чворове `call_expression` са функцијом `make`. Из листе аргумената чита именоване потомке: ако је први аргумент `slice_type`, а други је литерал `0` и нема трећег аргумента (капацитет), генерише се дијагностика.

```
if func_name == "make" && named_children.len() == 2 {
    let is_slice = named_children[0].kind() == "slice_type";
    let len_zero = named_children[1]
        .utf8_text(source.as_bytes()).ok()
        .as_deref() == Some("0");
    if is_slice && len_zero {
        diagnostics.push(/* ... */);
    }
}
```

Листинг 34: Детекција `make([]T, 0)` без капацитета

5.2.8 Празан литерал уместо нил исечка (nil_empty_slice)

Исечак (енг. *slice*) у Go-у је тројка: показивач на позадински низ, дужина и капацитет. Нил исечак (`var s []T`) је исечак код кога је показивач `nil`, а дужина и капацитет су нула. Ненил празан исечак (`[]T{}`) такође има дужину и капацитет нула, али показивач указује на стварну (иако празну) меморијску локацију.

У пракси ова разлика ретко утиче на понашање програма: оба облика прихватају `append`, оба имају `len` нула, оба се могу преиначити у петљи. Значајна разлика се јавља при серијализацији у JSON формат: нил исечак се серијализује као `null`, а ненил празан исечак као `[]`. Ако API позивалац очекује поље и добије `null`, може доћи до грешке на клијентској страни. Поред тога, `[]T{}` изазива мању алокацију коју Go-ов прикупљач смећа мора касније да обради. Из тих разлога, у Go заједници важи конвенција да је нил исечак пожељнији подразумевани облик кад елементи нису унапред познати [1].

```
s := []int{}    // ненил, нулта дужина; JSON → []
var s []int     // нил исечак, пожељнији; JSON → null
```

Листинг 35: Разлика између `[]T{}` и нил исечка

Правило проналази чворове `composite_literal` код којих је `type` поље `slice_type`, а `value` поље је `literal_value` чвор са нулом именованих потомака, тј. празним телом литерала.

```
if node.kind() == "composite_literal" {
    let is_slice = type_node.kind() == "slice_type";
    let is_empty = value_node.kind() == "literal_value"
        && value_node.named_child_count() == 0;
    if is_slice && is_empty { diagnostics.push(/* ... */); }
}
```

Листинг 36: Детекција празног литерала исечка

5.2.9 Провера исечка наспрам нил (slice_empty_check)

Исечак у Go-у може бити у три стања: нил (нису иницијализован), ненил и празан (иницијализован, али без елемената) и ненил и непразан (садржи елементе). Оператор `== nil` разликује само прво стање од преостала два: враћа `true` искључиво за нил исечак. Ненил празан исечак (`[]int{}` или `make([]int, 0)`) иницијализован је у меморији и тест `s == nil` враћа за њега `false`, иако је семантички такође “празан”.

Последица је да код попут `if s == nil { ... }` не обрађује сва могућа “празна” стања исечка. Робуснији образац је `len(s) == 0`, јер функција `len` враћа нулу и за нил и за ненил празан исечак [1].

```
var s []int
if s == nil { ... } // не покрива []int{}
if len(s) == 0 { ... } // исправно
```

Листинг 37: Поређење исечка са `nil` насупрот провере дужине

Правило је двофазно. У првој фази прикупљају се имена свих идентификатора чији тип је `slice_type` (из `var_declaration`) или чија је десна страна доделе иницијализација исечка. У другој фази обилазе се чворови `binary_expression` са операторима `==` или `!=` у чијем је операнду `nil`. Ако је суседни операнд идентификатор из скупа познатих исечака, пријављује се дијагностика.

```
let slice_side = match (&left, &right) {
  (Some(l), Some(r)) if r.kind() == "nil" => Some(*l),
  (Some(l), Some(r)) if l.kind() == "nil" => Some(*r),
  _ => None,
};
if let Some(node) = slice_side {
  if slice_vars.contains(name) { diagnostics.push(/* ... */); }
}
```

Листинг 38: Детекција поређења познатог исечка са `nil`

5.2.10 Копирање у исечак нулте дужине (`slice_copy`)

Уграђена функција `copy(dst, src)` копира `min(len(dst), len(src))` елемената. Ако одредишни исечак има нулту дужину, ниједан елемент неће бити копиран иако позив не пријављује грешку [1].

```
var dst []int
copy(dst, src) // dst је нил, len(dst)==0, ништа се не копира
```

Листинг 39: Копирање у исечак нулте дужине

У првој фази прикупљају се имена `var_declaration` исечака без иницијализатора (нил исечак, нулта дужина). У другој фази обилазе се позиви функције `copy`. Ако је први аргумент (`dst`) или нил идентификатор из прве фазе, или позив `make([]T, 0)` без капацитета, пријављује се дијагностика.

```
let is_problematic = match dst.kind() {
  "identifier" => nil_vars.contains(name),
  _ => self.is_zero_length_make(dst, source),
};
if is_problematic { diagnostics.push(/* ... */); }
```

Листинг 40: Провера одредишног аргумента позива `copy`

5.2.11 Нежељени ефекат додавања у подисечак (`slice_append_side_effect`)

Исечак у Go-у је прозор у позадински низ (енг. *backing array*): чува показивач на тај низ, дужину видљивог прозора и капацитет (колико места постоји у позадинском низу од почетка прозора). Оператор исецања `s[low:high]` ствара нови исечак над истим позадинским низом, само са другачијим почетком и дужином. Кључно је да оба исечка, оригинал и подисечак, деле исти позадински низ у меморији.

Функција `append` додаје елемент на крај исечка. Ако постоји слободан простор у капацитету (тј. `len < cap`), нови елемент се записује директно у позадински низ, не праве се копије. Управо ту лежи проблем: ако подисечак `s[1:3]` има капацитет 3 (јер оригинал иза индекса 3 има места), `append` ће нови елемент уписати на позицију `s[3]` у позадинском низу, тиме мењајући оригинални исечак без икаквог упозорења. Ово је особито опасно кад се подисечак и оригинал користе у различитим деловима кода [1].

Решење је тзв. трећи индекс: `s[low:high:high]` изричито поставља капацитет подисечка на `high - low`, тј. на тачно његову дужину. При следећем `append` капацитет је исцрпљен па Go мора да алоцира нови позадински низ, чиме се прекида дељење меморије.

```
s := []int{1, 2, 3, 4}
sub := s[1:3]
sub = append(sub, 99) // може да промени s[3]
```

Листинг 41: Додавање у подисечак са дељеним позадинским низом

Правило проналази позиве функције `append` и проверава да ли је први аргумент чвор `slice_expression`. Ако `slice_expression` нема поље `capacity` (тј. није трећи индекс `s[low:high:max]`), пријављује се дијагностика. Трећи индекс изричито ограничава капацитет подисечка и тиме спречава нежељени ефекат.

```
if func == "append" {
    if let Some(first) = args.named_child(0) {
        if first.kind() == "slice_expression"
            && first.child_by_field_name("capacity").is_none()
        {
            diagnostics.push(/* ... */);
        }
    }
}
```

Листинг 42: Детекција додавања у двоиндексни подисечак

5.2.12 Измена копије у `range` петљи (`range_copy_modification`)

Вредносна променљива `range` петље (`for i, v := range slice`) је копија елемента из исечка, а не референца. Додељивање пољу те копије не мења оригинал [1].

```
for _, item := range items {
    item.Value = 42 // мења копију, не оригинал
}
```

Листинг 43: Погрешна измена поља копије у `range` петљи

Правило прегледа `for_statement` чворове са `range_clause`. Из левог поља клаузуле чита се вредносна променљива (други именовани идентификатор, ако постоји и није `_`). Затим се тело петље претражује за `assignment_statement` чворове чији је леви операнд `selector_expression` чији `operand` одговара вредносној променљивој. Такви записи мењају копију и пријављују се.

```
fn check_selector_write(node, value_var, ...) {
    if node.kind() == "selector_expression" {
        let operand = node.child_by_field_name("operand")?;
        if operand.utf8_text(...)? == value_var {
            diagnostics.push(/* ... */);
        }
    }
}
```

Листинг 44: Детекција записа у поље вредносне `range` променљиве

5.2.13 Хватање адресе `range` променљиве (`range_pointer_capture`)

У верзијама Go-а пре 1.22, петљом `for i, v := range slice` компајлер декларише променљиве `i` и `v` само једном, пре прве итерације, и ажурира им вредности у свакој наредној итерацији. То значи да `i` и `v` у свим итерацијама заузимају исту меморијску локацију. Оператор адресе `&v` враћа адресу те локације, а не адресу конкретне вредности коју `v` тренутно носи.

Ако се та адреса чува у неком контејнеру (листи показивача, горутини и сл.), по завршетку петље сви сачувани показивачи показују на исто место у меморији — које садржи вредност из последње итерације.

Грешка се тешко уочава јер код у моменту чувања изгледа исправан, а погрешно понашање се манифестује тек при каснијем приступу сачуваним показивачима. Ово је посебно учестала грешка у конкурентном коду где горутине хватају &v пре него што петља заврши. Верзија Go 1.22 мења ово понашање тако да свака итерација добија нову променљиву, чиме грешка нестаје, али код написан за старије верзије остаје угрожен [1].

```
var ptrs []*int
for _, v := range nums {
    ptrs = append(ptrs, &v) // сви елементи показују на исти v
}
```

Листинг 45: Хватање адресе range променљиве

Правило прегледа for_statement чворове са range_clause и прикупља имена свих невређносних идентификатора на левој страни. Затим претражује тело петље за чворове unary_expression са оператором &, чији је операнд идентификатор из тог скупа.

```
if node.kind() == "unary_expression" {
    let op = node.child(0)?.utf8_text(...)? ;
    if op == "&" {
        let operand = node.child(1)?;
        if range_vars.iter().any(|v| v == name) {
            diagnostics.push(/* ... */);
        }
    }
}
```

Листинг 46: Детекција & оператора над range променљивом

5.2.14 Бројање бајтова уместо rune-ова (string_rune)

У Go-у, стринг је низ бајтова, а не низ знакова. Кодовање UTF-8, које Go подразумевано користи, представља ASCII знакове једним бајтом, али знакови других писама (ћирилица, кинески, јапански) или емоџи захтевају два до четири бајта. Због тога len(s) враћа број бајтова, а не број видљивих знакова: за стринг "Здраво" резултат је 12 (6 знакова по 2 бајта), а не 6.

Термин rune у Go-у означава Unicode кодну тачку (тип int32), тј. један видљиви знак без обзира на то колико бајтова заузима. Замена за len при бројању знакова је utf8.RuneCountInString(s).

Слична конфузија настаје при итерацији стринга. Облик for i := range s иде кроз rune позиције (у бајтовима) и i добија вредности 0, 2, 4, ... за ћирилични стринг. Ако се унутар петље приступи стрингу индексирањем s[i], резултат је бајт (uint8), не rune. Мултибајтни знак тако бива исечен на први бајт, и добија се погрешна вредност. Исправан облик је for i, r := range s где r носи цео rune [1].

Правило детектује конверзију string(небајт-израз) јер такав стринг скоро сигурно садржи мултибајтне знакове, па len над њим брои бајтове уместо знакова.

```
s := string(someBytes)
fmt.Println(len(s)) // број бајтова, не број знакова

for i := range s {
    fmt.Println(s[i]) // бајт, не rune
}
// Исправно:
fmt.Println(utf8.RuneCountInString(s))
for _, r := range s { fmt.Println(r) }
```

Листинг 47: Пгрешна употреба len и индексирање, и исправни облици

Правило је двофазно са два независна обрасца. У фази прикупљања подаци се деле у два скупа: `string_vars` (идентификатори стринг типа или стринг литерала) и `byte_string_vars` (идентификатори добијени конверзијом `string` (небајт-литерал)). Први образац пријављује позив `len` чији је аргумент члан `byte_string_vars`. Други образац проналази `for_statement` са `range_clause` над чланом `string_vars` где лева страна има тачно један невредносни идентификатор, па у телу петље тражи `index_expression` облика `s[i]`.

```
// Образац 1: len на бајт стрингу
if func == "len" && byte_string_vars.contains(arg_name) {
    diagnostics.push(/* ... */);
}
// Образац 2: s[i] у индексном range обиласку
if operand_ok && index_ok {
    diagnostics.push(/* ... */);
}
```

Листинг 48: Два обрасца детекције грешке rune/бајт конфузије

Циљ евалуације је провера тачности детекције за свако правило и идентификовање ограничења приступа заснованог на конкретном стаблу синтаксе. Евалуација је изведена у два корака: прво на наменски написаним тест фајловима са означеним позитивним и негативним случајевима, затим на стварном Go коду из производног пројекта.

6.1 Методологија

За свако од 21 правила написан је наменски тест фајл са следећом структуром. Позитивни случајеви (означени коментарима P1, P2, ...) представљају исечке кода за које правило треба да генерише дијагностику. Негативни случајеви (означени N1, N2, ...) представљају исечке за које правило не би требало да генерише дијагностику. Три фајла (`unused_variable.go`, `import_rules.go`, `integer_overflow.go`) садрже намерно некомпјабилан код и носе ознаку `//go:build ignore` јер Go компјалер одбацује неискоришћене варијабле, дупликате увоза и константе ван опсега. Ови фајлови се анализирају искључиво алатом `golint`.

За мерење тачности коришћене су следеће четири мере, приказане у табели 4:

Табела 4: Мере тачности

Скраћеница	Значење
ТП	Тачно позитивни: случајеви исправно пријављени
ТН	Тачно негативни: случајеви исправно непријављени
ЛП	Лажно позитивни: случајеви пријављени без стварног налаза
ЛН	Лажно негативни: стварни проблеми које алат није пронашао

6.2 Резултати евалуације

Табела 5 приказује резултате за сва 21 правила. Вредности у колонама ТП и ТН одговарају броју дијагностика генерисаних, односно исправно изостављених, за свако правило у његовом наменском тест фајлу.

Табела 5: Резултати евалуације за сва правила

Правило	Укупно	ТП	ТН	ЛП	ЛН
empty_block	7	4	3	0	0
unreachable_code	7	4	3	0	0
ignored_error	5	3	1	1	0
unused_variable	5	3	2	0	0
import_rules	3	2	1	0	0
variable_shadowing	5	3	2	0	0
defer_in_loop	6	3	3	0	0
utility_packages	1	1	0	0	0
no_any	8	5	3	0	0
generics_usage	4	1	3	0	0
mutex_embedding	4	2	2	0	0
package_collision	4	2	2	0	0
integer_overflow	7	4	3	0	0
slice_initialization	3	1	2	0	0
nil_empty_slice	5	2	3	0	0
slice_empty_check	5	3	2	0	0
slice_copy	4	2	2	0	0
slice_append_side_effect	4	2	2	0	0
range_copy_modification	4	2	2	0	0
range_pointer_capture	5	2	3	0	0
string_rune	5	2	3	0	0

Алат је укупно анализиран на 101 означеном случају. Од тога је тачно детектовао 53 позитивна и 47 негативних случајева, уз 1 лажно позитивни налаз и без иједног лажно негативног. Прецизност (однос ТП наспрам свих позитивних налаза) износи 98,1%, а осетљивост (однос ТП наспрам свих стварних позитивних случајева) износи 100%.

6.3 Поређење са постојећим алатима

Исти скуп фајлова покренут је и кроз три алата описана у поглављу 2: go vet, staticcheck и revive. Фајлови са ознаком //go:build ignore (unused_variable.go, import_rules.go, integer_overflow.go) аутоматски се прескачу свим Go-свесним алатима, а поређење покривености приказано је у табели 6.

Табела 6: Поређење покривености са постојећим алатима. ✓ = сви позитивни случајеви; 3/4, 2/4 = делимична покривеност; – = без детекције; ○ = фајл прескочен

Правило	golint	go vet	staticcheck	revive
empty_block	✓	–	–	3/4
unreachable_code	✓	✓	–	3/4
ignored_error	✓	–	–	–
unused_variable	✓	○	○	○
import_rules	✓	○	○	○
variable_shadowing	✓	–	–	–
defer_in_loop	✓	–	–	–
utility_packages	✓	–	–	–
no_any	✓	–	–	–
generics_usage	✓	–	–	–
mutex_embedding	✓	–	–	–
package_collision	✓	–	–	–
integer_overflow	✓	○	○	○
slice_initialization	✓	–	–	–
nil_empty_slice	✓	–	–	–
slice_empty_check	✓	–	–	–
slice_copy	✓	–	–	–
slice_append_side_effect	✓	–	–	–
range_copy_modification	✓	–	–	–
range_pointer_capture	✓	–	–	–
string_rune	✓	–	–	–

Међу постојећим алатима, само два правила имају делимично преклапање. Правило `unreachable_code` детектује `go vet` (свих 4/4 позитивних случајева), а `revive` 3 од 4 (пропушта случај после `panic`). Правило `empty_block` делимично покрива `revive` (3/4, без испразног тела функције); `staticcheck` не пријављује ниједан случај у самосталном покретању. Преосталих 19 правила не покрива ниједан испитани алат. Фајлови за правила `unused_variable`, `import_rules` и `integer_overflow` садрже намерно некомпјабилан код и носе ознаку `//go:build ignore`, услед чега их сви постојећи алати прескачу. Алат `golint` омогућава њихову анализу независно од компјлера.

Важно је напоменути да ово поређење не доказује да је `golint` бољи алат од постојећих. Евалуација је намерно ограничена на правила која су имплементирана у `golint`-у, па резултати само показују да `golint`

детектује управо оне обрасце за које је пројектован. Постојећи алати покривају знатно шири спектар провера које нису обухваћене овом евалуацијом: на пример, `staticcheck` нуди преко 150 засебних провера које обухватају семантичке грешке, deprecated API-је и неефикасне конструкције [3], а `go vet` детектује читав низ класа грешака везаних за позиве стандардне библиотеке. `golint` и постојећи алати нису директни конкуренти, него комплементарни алати: `golint` имплементира скуп правила инспирисаних добрим праксама из литературе која нису покривена у постојећим алатима.

6.4 Преостало ограничење: правило `ignored_error`

Правило `ignored_error` пријављује сваки бланк идентификатор `_` на левој страни доделе без обзира на тип вредности коју прима. На тест фајлу правило је, поред три тачна позитивна случаја игнорисања грешке, пријавило и случај

```
first, _ := returnsTwoStrings() // _ игнорише стринг, не error
```

Листинг 49: Лажно позитивни случај за правило `ignored_error`

где `_` прима стринг, а не вредност типа `error`. Ово лажно позитивно јавља се зато што КСС не садржи информације о типовима. Из овог разлога правило не може да разликује `error` тип од осталих типова. Потпуно решење захтевало би приступ информацијама о типовима из компајлера, нпр. преко `go/types` пакета, чиме би се напустио приступ независан од Go алатног ланца. Ово ограничење свесно је прихваћено у дизајну алата.

6.5 Евалуација на стварном коду

Алат је покренут на фајлу из стварног производног пројекта. Go HTTP серверу са приближно 200 линија кода за управљање паметним домом. Фајл садржи четири HTTP руковаоца (енг. *handler*), логику аутентификације, PostgreSQL упите и WebSocket сесије. Алат је пронашао пет налаза приказаних на листингу 50.

```
handlers_test.go:24:2 [mutex_embedding] Embedded `sync.RWMutex` in struct. This
exposes locking methods to the public API.
handlers_test.go:28:57 [no_any] Avoid using `any`, use a concrete type instead
handlers_test.go:65:3 [variable_shadowing] Variable `resp` shadows variable from
outer scope
handlers_test.go:76:2 [ignored_error] Error value assigned to blank identifier
handlers_test.go:187:25 [no_any] Avoid using `any`, use a concrete type instead
```

Листинг 50: Излаз алата на фајлу `handlers_test.go`

Налаз на линији 24 је тачан: анонимна структура која чува сесионе токене угради `sync.RWMutex` без имена поља, чиме методе `Lock` и `Unlock` постају видљиви јавни API. Налаз правила `no_any` на линијама 28 и 187 такође је тачан: функција `writeJSON` прима параметар типа `any` за тело HTTP одговора. Налаз на линији 65 правилно детектује декларацију `resp` унутар `if` блока у функцији `MotionTrigger`, која засенчује истоимену декларацију у спољашњем телу функције.

Налаз на линији 76 представља лажно позитивни случај: `_`, `err = h.DB.Exec(...)` занемарује повратну вредност типа `pgconn.CommandTag` (број промењених редова), а не вредност типа `error`. Грешка `err` је исправно прикупљена, али правило не може да разликује ова два повратна аргумента без информација о типовима.

Евалуација на стварном коду потврдила је два закључка. Прво, правила заснована на структурним обрасцима (уграђени мутекс, употреба `any`, засенчење опсега) раде поуздано на производном коду. Друго, преостало ограничење правила `ignored_error` је мало по обиму и предвидиво по природи.

У раду је развијен алат за статичку анализу Go изворног кода назван golint. Алат је имплементиран у програмском језику Rust и заснован је на библиотеци Tree-sitter, која производи конкретно стабло синтаксе директно из изворног текста. Кључна карактеристика алата је независност од Go компајлерске инфраструктуре: сваки синтаксно исправан Go фајл може бити анализиран без претходног компајлирања пројекта.

Имплементирано је укупно 21 правило подељено у два скупа. Седам правила добре праксе обухватају детекцију празних блокова, недостижног кода, игнорисаних грешака, некоришћених променљивих, проблема са увозима, засенчења опсега и погрешне употребе defer унутар петље. 14 правила инспирисана су књигом *100 Go Mistakes* [1] и покривају честе грешке везане за исечке, генерике, мутексе и Уникод.

Евалуација на 101 означеном тест случају показала је прецизност од 98,1% и осетљивост од 100%, уз свега 1 лажно позитивни налаз. Евалуација на стварном производном коду потврдила је поузданост структурних правила у пракси.

Главно ограничење приступа заснованог на конкретном стаблу синтаксе је одсуство информација о типовима. Ово ограничење утиче на правило `ignored_error`, које не може да разликује вредност типа `error` од осталих типова. Потпуно решење захтевало би интеграцију са `go/types` пакетом из Go инфраструктуре, чиме би се напустила независност од алатног ланца.

Могући правци даљег развоја укључују: подршку за конфигурисање правила из датотеке, додавање правила инспирисаних другим изворима о добрим праксама у Go програмирању, паралелно анализирање фајлова ради бољих перформанси на великим пројектима, као и опциону интеграцију са Go алатним ланцем ради отклањања поменутог ограничења.

Списак листинга

Листинг 1	КСС структура Tree-sitter парсера за кратку Go функцију	6
Листинг 2	Дефиниција аргумената употребом <code>clap</code> библиотеке	7
Листинг 3	Архитектура алата за статичку анализу	9
Листинг 4	Дефиниција <code>Rule</code> трејта	9
Листинг 5	Формат излазне дијагностике	11
Листинг 6	Пример излаза алата	11
Листинг 7	Пример празног блока кода	13
Листинг 8	Провера броја именованих потомака чвора <code>block</code>	13
Листинг 9	Пример недостижног кода	13
Листинг 10	Линеарни пролаз кроз листу наредби са заставицом	14
Листинг 11	Тихо игнорисање грешке бланк идентификатором	14
Листинг 12	Провера левих операнада доделе на бланк идентификатор	14
Листинг 13	Пример некоришћене променљиве	14
Листинг 14	Двофазна детекција некоришћених променљивих	15
Листинг 15	Дупликат и некоришћен увоз	15
Листинг 16	Провера дупликата и некоришћених увоза	15
Листинг 17	Пример засенчења променљиве	16
Листинг 18	Провера засенчења претраживањем стека опсега	16
Листинг 19	Пример погрешне употребе <code>defer</code> унутар петље	16
Листинг 20	Рекурзивна проверка <code>defer</code> унутар <code>for_statement</code> чвора	17
Листинг 21	Пример генеричког имена пакета	17
Листинг 22	Листа забрањених имена пакета	17
Листинг 23	Употреба <code>any</code> и <code>interface{}</code>	17
Листинг 24	Детекција <code>any</code> и <code>interface{}</code>	17
Листинг 25	Позив методе на генеричком параметру типа	18
Листинг 26	Детекција позива методе на генеричком аргументу	18
Листинг 27	Уграђени наспрам именованог <code>sync.Mutex</code>	18
Листинг 28	Детекција уграђеног мутекса по одсуству имена поља	19
Листинг 29	Локална променљива прекрива увезени пакет	19
Листинг 30	Провера нових декларација наспрам скупа увезених имена	19
Листинг 31	Целобројни литерал изван опсега типа	19
Листинг 32	Табела граница целобројних типова	19
Листинг 33	Иницијализација исечка без капацитета	20
Листинг 34	Детекција <code>make([]T, 0)</code> без капацитета	20
Листинг 35	Разлика између <code>[]T{}</code> и нил исечка	20
Листинг 36	Детекција празног литерала исечка	20
Листинг 37	Поређење исечка са <code>nil</code> насупрот провере дужине	21
Листинг 38	Детекција поређења познатог исечка са <code>nil</code>	21
Листинг 39	Копирање у исечак нулте дужине	21
Листинг 40	Провера одредишног аргумента позива <code>copy</code>	21
Листинг 41	Додавање у подисечак са дељеним позадинским низом	22
Листинг 42	Детекција додавања у двоиндексни подисечак	22
Листинг 43	Погрешна измена поља копије у <code>range</code> петљи	22
Листинг 44	Детекција записа у поље вредносне <code>range</code> променљиве	22

Листинг 45	Хватање адресе <code>range</code> променљиве	23
Листинг 46	Детекција & оператора над <code>range</code> променљивом	23
Листинг 47	Погрешна употреба <code>len</code> и индексирање, и исправни облици	23
Листинг 48	Два обрасца детекције грешке <code> rune</code> /бајт конфузије	24
Листинг 49	Лажно позитивни случај за правило <code>ignored_error</code>	28
Листинг 50	Излаз алата на фајлу <code>handlers_test.go</code>	28

Списак табела

Табела 1	Поређење алата за статичку анализу Go кода	4
Табела 2	Општа правила добре праксе	10
Табела 3	Правила инспирисана књигом <i>100 Go Mistakes and How to Avoid Them</i>	11
Табела 4	Мере тачности	25
Табела 5	Резултати евалуације за сва правила	26
Табела 6	Поређење покривености са постојећим алатима. ✓ = сви позитивни случајеви; 3/4, 2/4 = делимична покривеност; – = без детекције; ○ = фајл прескочен	27

Списак коришћених скраћеница

Скраћеница	Опис
ACC	Abstract Syntax Tree (апстрактно стабло синтаксе)
API	Application Programming Interface (апликациони програмски интерфејс)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
KCC	Concrete Syntax Tree (конкретно стабло синтаксе)
ЛН	Лажно негативни
ЛП	Лажно позитивни
Н/А	Не примењује се
ТН	Тачно негативни
ТП	Тачно позитивни

Списак коришћених појмова

Појам	Објашњење
Власништво	Rust концепт управљања меморијом у ком компајлер прати власника сваке вредности и аутоматски ослобађа меморију када власник изађе из опсега, без употребе сакупљача смећа.
Дијагностика	Порука коју алат генерише при откривању проблема у анализираном коду, која садржи назив правила, опис налаза и тачну позицију у изворном фајлу (ред и колона).
Засенчење	Поновна декларација варијабле истог имена у унутрашњем опсегу, чиме нова варијабла прикрива спољашњу; може да прикрије неочекивану промену типа или вредности.
Линтер	Алат за статичку анализу изворног кода који аутоматски открива потенцијалне грешке, кршења стилских конвенција и лоше праксе без покретања програма.
Опсег	Регион у изворном коду у ком је варијабла или симбол видљив и употребљив.
Осетљивост	Однос тачно позитивних налаза наспрам свих стварних позитивних случајева у тест скупу; мери колико је проблема алат успео да открије.
Парсирање	Процес превођења изворног текста у стабло синтаксе које структурно представља програм.
Прецизност	Однос тачно позитивних налаза наспрам свих пријављених налаза; мери колики је удео пријављених проблема заиста исправан.
Статичка анализа	Техника испитивања изворног кода без његовог извршавања, са циљем откривања грешака и лоших пракси.
Трејт	Rust концепт апстрактног интерфејса; дефинише скуп метода које тип мора да имплементира, аналогно интерфејсима у другим програмским језицима.

Биографија

Овде написати своју кратку биографију.

Литература

- [1] T. Harsanyi, *100 Go Mistakes and How to Avoid Them*. Manning Publications, 2022.
- [2] “go vet — Go Command Documentation.” Accessed: June 20, 2026. [Online]. Доступно на <https://pkg.go.dev/cmd/vet>
- [3] “Staticcheck — The Advanced Go Linter.” Accessed: June 20, 2026. [Online]. Доступно на <https://staticcheck.dev/>
- [4] “revive — Fast, Configurable, Extensible Linter for Go.” Accessed: June 20, 2026. [Online]. Доступно на <https://revive.run/>
- [5] “golangci-lint — Fast Go Linters Runner.” Accessed: June 20, 2026. [Online]. Доступно на <https://golangci-lint.run/>
- [6] N. D. Matsakis and F. S. K. II, “The Rust Language,” in *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, ACM, 2014. doi: [10.1145/2663171.2663188](https://doi.org/10.1145/2663171.2663188).
- [7] “Rust Programming Language.” Accessed: June 20, 2026. [Online]. Доступно на <https://rust-lang.org/>
- [8] S. Klabnik and C. Nichols, *The Rust Programming Language*. No Starch Press, 2019. [Online]. Доступно на <https://doc.rust-lang.org/book/>
- [9] “Tree-sitter — An Incremental Parsing System for Programming Tools.” Accessed: June 20, 2026. [Online]. Доступно на <https://tree-sitter.github.io/tree-sitter/>
- [10] “clap — A Full-Featured Command Line Argument Parser for Rust.” Accessed: June 22, 2026. [Online]. Доступно на <https://docs.rs/clap/latest/clap/>